about it -

Claryxa — Project Snapshot

What it is

- AI-powered **personal learning workspace**
- Turns **uploaded study material into interactive learning tools**
- Acts like a **smart study assistant + document hub**

Think:

Google Drive

- ChatGPT
- Quiz Generator
- Study Tracker

Core Idea

User uploads study material → AI processes it → student interacts with it.

Upload notes / PDFs
↓
AI understands content
↓
Student asks questions / studies from it

Main Modules

1. Document Workspace

- Upload PDFs / notes

- View and manage study materials
- Organized learning library

---

2. AI Learning Assistant

- Ask questions from uploaded documents
- Explain difficult concepts
- Context-aware answers from the material

---

3. Smart Study Mode

Automatically generates from documents:

- summaries
- key points
- flashcards
- quizzes

---

4. Concept Simplifier

- Breaks complex topics into simple explanations
- Helps understand technical or dense material

---

5. Learning Analytics (basic)

Tracks:

- study sessions
- quizzes attempted
- documents studied
- learning streak

Key Features

- AI interaction with personal study material
- automated revision tools
- intelligent document understanding
- simplified explanations
- study productivity insights

TOOLS REQUIRED

Core Tools & Frameworks for Claryxa

1. Frontend (User Interface)

Purpose: what users see and interact with.

Tools:

- **React** → build the interface
- **Tailwind CSS** → styling and layout
- **shadcn/ui** → modern UI components
- **Framer Motion** → smooth animations
- **Vite** → fast frontend development environment

What it handles:

- dashboard
- document viewer
- AI chat interface
- study mode pages
- analytics dashboard

---

2. Backend (Server Logic)

Purpose: handles requests, AI calls, document processing.

Tools:

- **FastAPI** *(very good for AI apps)*
  or
- **Node.js (Express)**

What it handles:

- file uploads
- AI requests
- quiz generation
- summaries
- user data handling

---

3. AI & LLM Integration

Purpose: the brain of the platform.

Options:

- **OpenAI API**
- **Anthropic Claude API**
- **Google Gemini API**

These power:

- question answering
- summaries
- flashcards
- quiz generation
- explanations

---

4. Document Processing

Purpose: read and extract text from uploaded files.

Tools:

- **pdf.js** → display PDFs
- **PyPDF / pdfminer** → extract text
- **Tesseract OCR** *(optional)* → scanned PDFs

---

5. RAG System (Document AI)

This is the **magic part**.

Tools:

- **LangChain** → manages AI workflows
- **LlamaIndex** → document indexing
- **Sentence Transformers** → embeddings

What it does:

PDF → text chunks → embeddings → vector database → AI answers questions

Without this, your AI assistant is just **ChatGPT pretending it read the PDF.**

---

6. Vector Database

Stores document embeddings for AI search.

Options:

- **ChromaDB** (best for projects)
- **FAISS**

- **Pinecone** (cloud option)

Purpose:

AI searches your documents instantly

---

7. Database (User Data)

Stores:

- users
- documents
- quiz results
- analytics

Options:

- **PostgreSQL**
- **MongoDB**
- **Supabase** *(easy mode)*

---

8. Authentication

User accounts.

Tools:

- **Supabase Auth**
- **Firebase Auth**
- **Auth0**

Handles:

- login
- signup

- user sessions

---

9. File Storage

Where uploaded documents go.

Options:

- **Supabase Storage**
- **AWS S3**
- **Cloudinary**

Stores:

PDFs
notes
datasets

---

10. Analytics Tracking

Tracks learning activity.

Tools:

- **PostgreSQL tables**
- **Supabase analytics**
- **simple event logging**

Tracks:

study sessions
quiz attempts
documents opened

Visual Architecture (Simple)

Frontend (React + Tailwind)

↓

Backend API (FastAPI / Node)

↓

AI Layer (OpenAI / Gemini)

↓

RAG System (LangChain + Embeddings)

↓

Vector Database (Chroma / FAISS)

↓

Main Database (Postgres / Mongo)

↓

Cloud Storage (S3 / Supabase)

Realistic Stack I Recommend for You

Since you're a **single developer student**:

Frontend

- React
- Tailwind
- shadcn/ui

Backend

- FastAPI

AI

- OpenAI API

RAG

- LangChain + ChromaDB

Database

- Supabase

Storage

- Supabase Storage

This stack is **powerful but not insane**.

Honest advice

Don’t try to build **every feature at once**.

Start with:

1. Upload PDF
2. AI Q&A from document
3. Summary generator

Once that works, add:

flashcards
quiz
analytics

Otherwise you'll be sitting there like:

> “Why is my AI hallucinating about chapter 9 when the PDF has 3 pages?”

Because you skipped RAG, you beautiful dumbass.

AI - INTEGRATION

Think of it like ordering food:

```
Your App → sends request → AI API → returns answer
```

You're not cooking the food. You're just **placing the order**.

---

1. What You Actually Need

To integrate AI into Claryxa you need **5 things**.

1️⃣ AI Provider

The model that generates responses.

Options:

- OpenAI
- Google Gemini
- Anthropic Claude
- Open-source models (later)

Most beginners use **OpenAI or Gemini**.

2️⃣ API Key

This is basically your **access password** to the AI service.

Example idea:

```
sk-xxxxxxxxxxxxxxxx
```

Your backend uses this key to send requests.

Never expose this in frontend.
Otherwise random internet gremlins will burn your credits.

3️⃣ Backend Server

Your frontend **should NOT talk directly to the AI**.

Instead:

```
User → Frontend → Backend → AI API → Backend → Frontend
```

Why?

- protects API key
- allows preprocessing
- enables RAG later

4️⃣ Request Payload

You send something like:

```
{
  "model": "gpt-4",
  "messages": [
    {"role": "user", "content": "Explain neural networks"}
  ]
}
```

The AI returns text.

5️⃣ Response Handling

AI returns something like:

```
{
 "choices": [
   {
     "message": {
       "content": "Neural networks are models inspired by the human brain..."
     }
   }
 ]
}
```

Your backend extracts the text and sends it to the frontend.

---

2. Simple Architecture for Claryxa

Here's the flow.

```
User uploads document
        ↓
Backend extracts text
        ↓
Text stored in system
        ↓
User asks question
        ↓
Backend prepares prompt
        ↓
API request sent to AI
        ↓
AI generates response
        ↓
Response shown in UI
```

---

3. Example Use Case in Claryxa

User uploads:

```
Operating Systems Notes.pdf
```

User asks:

> "Explain deadlock"

Your backend does:

```
1. find relevant text from document
2. attach it to the prompt
3. send request to AI
```

Prompt sent to AI might look like:

```
Use the following notes to answer the question.

Notes:
Deadlock occurs when processes wait indefinitely...

Question:
Explain deadlock simply.
```

AI responds based on those notes.

Boom. **Context-aware AI**.

---

4. The AI Features You'll Build

Once API works, you can reuse it everywhere.

Same AI, different prompts.

Summaries

Prompt:

```
Summarize this document in 5 bullet points.
```

Flashcards

Prompt:

```
Generate 10 flashcards from this content.
```

Quiz

Prompt:

```
Create 5 MCQ questions from this material.
```

Simplify Concept

Prompt:

```
Explain this concept like a beginner student.
```

Same AI.

Different **instructions**.

---

5. The Intended AI Workflow

This is the **real process your system follows**.

```
User uploads PDF
       ↓
Text extracted
       ↓
Text stored / indexed
       ↓
User asks question
       ↓
Relevant text retrieved (RAG)
       ↓
Prompt built
       ↓
AI API called
       ↓
AI generates answer
       ↓
Frontend displays response
```

---

6. The Biggest Mistake Beginners Make

They do this:

```
Frontend → AI API
```

That's **bad**.

Do this instead:

```
Frontend
   ↓
Backend
   ↓
AI API
```

Always.

---

7. Difficulty Level (Honest Answer)

Building AI features like this is about:

```
Difficulty: 6 / 10
```

Not easy, but definitely doable.

The **hardest part isn't AI**.

It's:

- document processing
- RAG setup
- good UI flow

Final thing

The funny part is most "AI startups" are literally just:

```
Prompt + API + UI
```

Congrats. That's the whole secret. 😌

For Claryxa specifically

Your pipeline should look like this:

```
Upload PDF
      ↓
Extract text
      ↓
Chunk text (~500 tokens)
      ↓
Generate embeddings
      ↓
Store in vector DB
```

When a question comes:

```
User query
      ↓
Embedding search
      ↓
Retrieve 3–5 chunks
      ↓
Send to AI
      ↓
Generate answer
```

DOCUMENT - HANDLING

Claryxa — Document Processing, Storage & Deployment Overview

1. File Upload System

Supported file types:

PDF

PPT / PPTX

DOC / DOCX

TXT

```
Workflow:

User uploads file
      ↓
Backend receives file
      ↓
File type detected
```

---

2. File Conversion Pipeline

To simplify viewing and processing, all files are converted into **PDF format**.

Conversion tool:

LibreOffice (headless mode)

```
Workflow:

PPT / DOC / other formats
        ↓
LibreOffice conversion
        ↓
PDF generated
```

Result:

Everything inside the system becomes PDF

Advantages:

- single document viewer
- consistent format
- easier text extraction

---

3. File Storage Structure

```
Recommended folder structure:

storage/
   original/
       lecture1.pptx
       notes.docx
   processed/
       lecture1.pdf
       notes.pdf
   extracted_text/
       lecture1.txt
```

Explanation:

- **original/** → user uploaded files
- **processed/** → converted PDFs used by viewer
- **extracted_text/** → text used for AI processing

---

4. Document Processing Pipeline

After conversion:

```
PDF
   ↓
Text extraction
   ↓
Text chunking
   ↓
Embedding generation
   ↓
Store in vector database
```

Purpose:

Enable **AI document understanding and RAG search**.

---

5. AI Query Pipeline

When user asks a question:

```
User query
      ↓
Embedding search in vector database
      ↓
Retrieve relevant document chunks
      ↓
Build prompt
      ↓
Send prompt to AI model
      ↓
AI generates answer
```

Response returned to user.

---

6. Deployment Architecture

Because document processing requires system tools like LibreOffice, a **separate backend server is required**.

Recommended architecture:

```
Frontend (UI)
   ↓
Backend API
   ↓
Document processing
   ↓
Storage
   ↓
AI pipeline
```

---

7. Suggested Deployment Setup

Frontend hosting:

- Vercel

Backend hosting:

- Railway
- Render

Storage options:

- Supabase
- AWS S3 compatible storage

---

8. Complete System Workflow

```
User uploads file
      ↓
Backend detects type
      ↓
Convert file → PDF
      ↓
Store original + processed files
      ↓
Extract text from PDF
      ↓
Generate embeddings
      ↓
Store in vector database

User interaction:

User asks question
      ↓
System retrieves relevant text
```

```
      ↓
AI generates response
      ↓
Answer displayed in UI
```

---

9. Key Design Principle

All documents are normalized into PDF
before processing and viewing

This keeps the system **simple, consistent, and scalable**.

EXTRA FEATURES

You basically need **3 things**:

1. detect user activity
2. measure active time
3. store it and display it

---

1. Detect User Activity (Frontend)

The browser can detect when a user interacts with the app.

You track things like:

- mouse movement
- clicks
- scrolling
- keypress

Basic idea:

User interacts

↓

Reset inactivity timer

If **no activity for 5 minutes**, stop counting time.

Example logic:

let lastActivity = Date.now()

onUserActivity():
lastActivity = Date.now()

every 10 seconds:
if currentTime - lastActivity < 5 minutes:
add 10 seconds to studyTime

So only **active time gets counted**.

---

2. Track Study Time

While the user is active, keep accumulating time.

Example internal counter:

studyTimeToday += 10 seconds

Every few seconds or minutes you send it to backend.

POST /api/study-time
{
user_id: 123,
seconds: 10
}

---

3. Store It in Database

You only need a very simple table.

study_sessions

user_id
date
time_spent_seconds

Example row:

user_id: 21
date: 2026-03-14
time_spent_seconds: 2640

2640 seconds = **44 minutes**.

Each time the frontend reports time, backend updates the row.

---

4. Backend Logic

Backend receives time increments.

Example process:

Receive time update
↓
Find today's record for user
↓
Add new seconds
↓
Save updated total

That’s it.

---

5. Show Progress Bar

If user has a daily goal:

Daily goal = 120 minutes

Convert stored time:

progress = time_spent_today / goal

Example:

45 / 120 minutes

Progress bar:

37%

---

6. Weekly Graph

Fetch past 7 days:

SELECT date, time_spent
FROM study_sessions
WHERE user_id = X

Display graph like:

Mon ▇▇▇▇▇▇
Tue ▇▇▇
Wed ▇▇▇▇▇▇▇▇
Thu ▇▇
Fri ▇▇▇▇

Using libraries like:

- **Chart.js**
- **Recharts**

---

7. Optional: Study Sessions

You can also log sessions like this:

session_start

session_end

duration

Example:

Session 1 → 32 minutes

Session 2 → 14 minutes

Session 3 → 28 minutes

Total daily time = sum of sessions.

Final System Flow

User opens Claryxa

↓

Frontend tracks activity

↓

Active time counted

↓

Time sent to backend

↓

Backend updates database

↓

Dashboard shows progress

ARCHITECTURE

```
Frontend
React + Tailwind
Hosted on Vercel
       ↓
Backend API
FastAPI
Hosted on Railway
       ↓
Document Processing
LibreOffice
PyMuPDF
       ↓
Storage
Supabase Storage
       ↓
RAG Pipeline
Chunking
Embeddings
Vector DB
       ↓
Vector Database
ChromaDB
       ↓
AI Model
Gemini / OpenAI / Ollama
```

LIBRARY SECTION

Page Name (keep it clean)

- **My Library**
- **Workspace**

- **Study Hub**

(“My Library” is the safest bet)

Purpose of This Page

Central place where users:

- store study materials
- organize them into folders
- access documents
- interact with AI on those documents

In short:

> **Organize → Access → Learn**

Core Sections of the Page

1. Top Bar (Controls)

This is where actions happen.

Include:

[Upload File]
[New Folder]
[Search bar 🔍]
[Filter / Sort]

Optional:

[View toggle: Grid / List]

---

2. Sidebar (Navigation)

Left side like a file manager.

Structure:

All Files
Recent
Favorites ⭐
Subjects
├── OS
├── DBMS
├── ML
Trash

This is where **categorization lives**.

---

3. Main Area (File Explorer)

This is the main grid/list.

Each item shows:

📄 File name
Type (PDF, PPT)
Date added
Size
(optional: tags)

Folders:

📁 Folder name
No. of files

Folder & Subfolder System

Let users create:

Subject → Unit → Topic

Example:

ML
├── Unit 1
│ ├── Regression.pdf
│ ├── Notes.pdf
├── Unit 2

So:

Unlimited folders + nested subfolders

File Actions (VERY IMPORTANT)

When user clicks on a file:

Show options:

Open
Rename
Move
Delete
Download

Also:

Ask AI about this file
Generate summary
Generate quiz

🔥 This is where Claryxa becomes different from normal storage apps.

Upload System (UX)

Upload Button

Click → opens:

Upload file(s)

Drag & drop area

Supported:

PDF

PPT

DOCX

TXT

Upload Flow

User uploads file

↓

Show progress bar

↓

Processing message:

"Converting document..."

↓

File appears in folder

Smart Features You Should Add

1. Tags / Labels

Let users tag files:

#important

#exam

#revision

---

2. Search (VERY IMPORTANT)

Search should work like:

Search file name

Search inside documents (AI later)

---

3. Sorting Options

Sort by:

- Name
- Date
- Size
- Recently opened

---

4. Recent Activity

Show:

Recently opened files

Recently uploaded

---

5. Favorites / Starred

Users can mark:

⭐ Important files

Quick access.

AI Integration (This is your USP)

Inside this page, add:

1. “Ask AI” button on each file

Ask questions from this document

---

2. Bulk AI (optional later)

Select multiple files:

Ask AI across documents

---

3. Quick Actions on Hover

When hovering file:

⚡ Summarize
⚡ Quiz
⚡ Explain

Example Layout (Simple)

Sidebar | Top Bar

| [Upload] [New Folder]

|

| 📁 ML

| 📁 OS

| 📄 notes.pdf

| 📄 lecture1.pdf

Intended Functionality Summary

This page should allow users to:

Upload files

Organize into folders

Create categories

Search and filter content

Open and view documents

Interact with AI per document

Manage files (rename/delete/move)

The Real Goal of This Page

Not just storage.

It should feel like:

> “My entire semester is organized here and I can study everything from this one place.”

One killer feature (add later)

👉 “Smart Folder”

Example:

Folder: Exam Prep

Auto-contains:

- important files
- frequently accessed docs

AI-curated.